

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1 ✓
Student Performance Analysis
— List + Tuple
10 marks

Q2 ✓
Customer Name Analysis —
String + List
10 marks

Q3 ✓
Product Inventory Analysis —
List + Tuple
10 marks

Q4 ✓
Text Analysis — String + List
10 marks

Q5 ✓
Employee Skill Analysis — Tuple
+ List + String
10 marks

Q6 ✓
Transaction Analysis — Tuple +
List
10 marks

Q7 ✓
Password Security Analysis —
String + List
10 marks

Q8 ✓
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9 ✓
Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10 ✓
Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Q1 Student Performance Analysis — List + Tuple

10 marks

A college stores the following student records:

```
students = [
    ("Arun", [78, 85, 92, 67]),
    ("Bala", [45, 67, 55, 49]),
    ("Charan", [90, 92, 88, 95]),
    ("Divya", [65, 72, 70, 68]),
    ("Esha", [88, 76, 91, 84])
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.
9. Identify the top two students.
10. Explain which parts of the structure are mutable and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Divya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

Your Code

```
1 students = [  
2     ("Arun", [78, 85, 92, 67]),  
3     ("Bala", [45, 67, 55, 49]),  
4     ("Charan", [90, 92, 88, 95]),  
5     ("Divya", [65, 72, 70, 68]),  
6     ("Esha", [88, 76, 91, 84])  
7 ]  
8  
9 # 1. Calculate total and average  
10 totals = {}  
11 averages = {}  
12  
13 for name, marks in students:  
14     total = sum(marks)  
15     average = total / len(marks)  
16     totals[name] = total  
17     averages[name] = average  
18  
19 print("Student Totals:")  
20 for name in totals:  
21     print(name, ":", totals[name])  
22  
23 print("\nStudent Averages:")  
24 for name in averages:  
25     print(name, ":", format(averages[name], ".2f"))  
26  
27 # Class average  
28 class_average = sum(averages.values()) / len(averages)  
29  
30 # 2. Highest total  
31 highest = max(totals, key=totals.get)  
32 print("\nHighest Total:", highest)  
33  
34 # 3. Lowest average  
35 lowest = min(averages, key=averages.get)  
36 print("Lowest Average:", lowest)  
37  
38 # 4. Students
```

Input

stdin input

Output

```
Student Totals:  
Arun : 322  
Bala : 216  
Charan : 365  
Divya : 275  
Esha : 339
```



Run



Save

An e-commerce company receives customer names in inconsistent formats:

```
names = [  
    "Arun Kumar ",  
    "ARUN KUMAR",  
    " arun kumar",  
    "Bala Krishnan",  
    "BALA KRISHNAN ",  
    "Charan",  
    " charan ",  
    "DIVYA DEVI",  
    "Divya Devi"  
]
```

Analyze the customer data.

Tasks

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.
6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.
8. Count the number of characters in each normalized name.
9. Sort the unique names alphabetically.
10. Determine which customer name occurs most frequently.

Expected Output:

Normalized:

```
['arun kumar', 'arun kumar', 'arun kumar',  
'bala krishnan', 'bala krishnan',  
'charan', 'charan',  
'divya devi', 'divya devi']
```

Unique Customers:

```
['arun kumar', 'bala krishnan', 'charan', 'divya devi']
```

Number of Unique Customers: 4

Duplicate Customers:

```
arun kumar  
bala krishnan  
charan  
divya devi
```

Names with Multiple Words:

```
arun kumar  
bala krishnan  
divya devi
```

Longest Name:

```
bala krishnan
```

Starts with A or D:

```
arun kumar  
divya devi
```

```

1 students = [
2     ('Arun', [78, 85, 92, 67]),
3     ('Bala', [45, 67, 55, 49]),
4     ('Charan', [90, 92, 88, 95]),
5     ('Divya', [65, 72, 70, 68]),
6     ('Esha', [88, 76, 91, 84])
7 ]
8
9 # 1. Calculate total and average
10 totals = {}
11 averages = {}
12
13 for name, marks in students:
14     total = sum(marks)
15     average = total / len(marks)
16     totals[name] = total
17     averages[name] = average
18
19 print('Student Totals:')
20 for name in totals:
21     print(name, ': ', totals[name])
22
23 print('\nStudent Averages:')
24 for name in averages:
25     print(name, ': ', format(averages[name], '.2f'))
26
27 # Class average
28 class_average = sum(averages.values()) / len(averages)
29
30 # 2. Highest total
31 highest = max(totals, key=totals.get)
32 print('\nHighest Total:', highest)
33
34 # 3. Lowest average
35 lowest = min(averages, key=averages.get)
36 print('Lowest Average:', lowest)
37
38 # 4. Students above class average
39 above_class = [name for name in averages if averages[name] > class_average]
40 print('Above Class Average:', ', '.join(above_class))
41
42 # 5. Students with 3 or more marks above 80
43 three_above_80 = []
44
45 for name, marks in students:
46     count = sum(mark > 80 for mark in marks)
47     if count >= 3:
48         three_above_80.append(name)
49
50 print('\nStudents with 3 or more marks above 80:')
51 print(', '.join(three_above_80))
52
53 # 6. Highest mark in each subject
54 print('\nSubject-wise Highest:')
55 for i in range(4):
56     highest_mark = max(marks[i] for name, marks in students)
57     print('Subject', i + 1, ': ', highest_mark)
58
59 # 7. Students who scored below 50 in any subject
60 below_50 = []
61
62 for name, marks in students:
63     if any(mark < 50 for mark in marks):
64         below_50.append(name)
65
66 print('\nStudents with a mark below 50:')
67 print(', '.join(below_50))
68
69 # 8. Sort students by average in descending order
70 sorted_students = sorted(
71     averages.items(),
72     key=lambda x: x[1],
73     reverse=True
74 )
75
76 # 9. Top two students
77 print('\nTop 2:')
78 for name, average in sorted_students[:2]:
79     print(name)
80
81 # 10. Mutable and immutable
82 print('\nMutable: List')
83 print('Immutable: Tuple')

```

```

34 # 3. Lowest average
35 lowest = min(averages, key=averages.get)
36 print("Lowest Average:", lowest)
37
38 # 4. Students above class average
39 above_class = [name for name in averages if averages[name] > class_aver
40 print("Above Class Average:", ", ".join(above_class))
41
42 # 5. Students with 3 or more marks above 80
43 three_above_80 = []
44
45 for name, marks in students:
46     count = sum(mark > 80 for mark in marks)
47     if count >= 3:
48         three_above_80.append(name)
49
50 print("\nStudents with 3 or more marks above 80:")
51 print(", ".join(three_above_80))
52
53 # 6. Highest mark in each subject
54 print("\nSubject-wise Highest:")
55 for i in range(4):
56     highest_mark = max(marks[i] for name, marks in students)
57     print("Subject", i + 1, ":", highest_mark)
58
59 # 7. Students who scored below 50 in any subject
60 below_50 = []
61
62 for name, marks in students:
63     if any(mark < 50 for mark in marks):
64         below_50.append(name)
65
66 print("\nStudents with a mark below 50:")
67 print(", ".join(below_50))
68
69 # 8. Sort students by average in descending order
70 sorted_students = sorted(
71     averages.items(),
72     key=lambda x: x[1],
73     reverse=True
74 )
75
76 # 9. Top two students
77 print("\nTop 2:")
78 for name, average in sorted_students[:2]:
79     print(name)
80
81 # 10. Mutable and immutable
82 print("\nMutable: List")
83 print("Immutable: Tuple")

```

Input

stdin input

Output

```

Student Totals:
Arun : 322
Bala : 216
Charan : 365
Divya : 275
Esha : 339

```

Run

Save

A supermarket stores product information as:

```
products = [  
    ("Laptop", 5, 55000),  
    ("Mouse", 25, 700),  
    ("Keyboard", 8, 1200),  
    ("Monitor", 12, 15000),  
    ("Printer", 4, 12000),  
    ("Headset", 15, 2500)  
]
```

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory.

Tasks

1. Calculate the inventory value of every product.
2. Calculate the total inventory value.
3. Identify products whose quantity is less than 10.
4. Identify products whose price is greater than ₹10,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than ₹50,000 to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 37500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
1 products = [  
2     ("Laptop", 5, 55000),  
3     ("Mouse", 25, 700),  
4     ("Keyboard", 8, 1200),  
5     ("Monitor", 12, 15000),  
6     ("Printer", 4, 12000),  
7     ("Headset", 15, 2500)  
8 ]  
9  
10 # 1. Inventory value of every product  
11 print('Inventory Values:')  
12 values = {}  
13  
14 for name, quantity, price in products:  
15     value = quantity * price  
16     values[name] = value  
17     print(name, ': ', value)  
18  
19 # 2. Total inventory value  
20 total_value = sum(values.values())  
21 print('\nTotal Inventory Value:', total_value)  
22  
23 # 3. Products with quantity less than 10  
24 print('\nLow Stock:')  
25 for name, quantity, price in products:  
26     if quantity < 10:  
27         print(name)  
28  
29 # 4. Products with price greater than 10000
```

Your Code

```
1 products = [  
2     ('Laptop', 5, 55000),  
3     ('Mouse', 25, 700),  
4     ('Keyboard', 8, 1200),  
5     ('Monitor', 12, 15000),  
6     ('Printer', 4, 12000),  
7     ('Headset', 15, 2500)  
8 ]  
9  
10 # 1. Inventory value of every product  
11 print('Inventory Values:')  
12 values = {}  
13  
14 for name, quantity, price in products:  
15     value = quantity * price  
16     values[name] = value  
17     print(name, ': ', value)  
18  
19 # 2. Total inventory value  
20 total_value = sum(values.values())  
21 print('\nTotal Inventory Value:', total_value)  
22  
23 # 3. Products with quantity less than 10  
24 print('\nLow Stock:')  
25 for name, quantity, price in products:  
26     if quantity < 10:  
27         print(name)  
28  
29 # 4. Products with price greater than 10000  
30 print('\nPrice > 10000:')  
31 for name, quantity, price in products:  
32     if price > 10000:  
33         print(name)  
34  
35 # 5. Most expensive product  
36 expensive = max(products, key=lambda x: x[2])  
37 print('\nMost Expensive:')  
38 print(expensive[0])  
39  
40 # 6. Product with highest inventory value  
41 highest_value = max(values, key=values.get)  
42 print('\nHighest Inventory Value:')  
43 print(highest_value)  
44  
45 # 7. Sort products based on price  
46 print('\nSorted by Price:')  
47 for name, quantity, price in sorted(products, key=lambda x: x[2]):  
48     print(name, ': ', price)  
49  
50 # 8. Sort products based on quantity  
51 print('\nSorted by Quantity:')  
52 for name, quantity, price in sorted(products, key=lambda x: x[1]):  
53     print(name, ': ', quantity)  
54  
55 # 9. Total quantity  
56 total_quantity = sum(quantity for name, quantity, price in products)  
57 print('\nTotal Quantity:', total_quantity)  
58  
59 # 10. Products contributing more than 50000  
60 print('\nInventory Value > 50000:')  
61 for name in values:  
62     if values[name] > 50000:  
63         print(name)
```

Input

stdin input

Output

```
Inventory Values:  
Laptop : 275000  
Mouse : 17500  
Keyboard : 9600  
Monitor : 180000  
Printer : 48000
```



Run



Save

A university receives the following feedback:

feedback = """

Python programming is easy to learn.

Python programming is powerful.

Learning Python requires practice.

Practice makes programming skills better.

"""

Analyze the text.

Tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

Your Code

```
1 feedback = """Python programming is easy to learn.  
2 Python programming is powerful.  
3 Learning Python requires practice.  
4 Practice makes programming skills better."""  
5  
6 # Convert to lowercase  
7 text = feedback.lower()  
8  
9 # Remove punctuation  
10 text = text.replace(".", "")  
11 text = text.replace(",", "")  
12  
13 # Convert into words  
14 words = text.split()  
15  
16 # Unique words  
17 unique = []  
18 for word in words:  
19     if word not in unique:  
20         unique.append(word)  
21  
22 print("Total Words:", len(words))  
23 print("Unique Words:", len(unique))  
24  
25 # Frequency  
26 print("\nFrequency of python:", words.count("python"))  
27 print("Frequency of programming:", words.count("programming"))  
28  
29 # Repeated words  
30 print("\nRepeated Words:")  
31 for word in unique:  
32     if words.count(word) > 1:  
33         print(word)  
34  
35 # Longest word  
36 longest = max(words, key=len)  
37 print("\nLongest Word:")  
38 print(longest)  
39  
40 # Words containing p  
41 print("\nWords containing 'p':")  
42 for word in unique:  
43     if "p" in word:  
44         print(word)  
45  
46 # Percentage of unique words  
47 percentage = (len(unique) / len(words)) * 100  
48 print("\nPercentage of Unique Words:", round(percentage, 2), "%")
```

Input

stdin input

Output

```
Total Words: 19  
Unique Words: 13  
  
Frequency of python: 3  
Frequency of programming: 3
```

An organization stores:

```
employees = [  
    ("E101", "Arun", ["Python", "SQL", "ML"]),  
    ("E102", "Bala", ["Java", "SQL", "HTML"]),  
    ("E103", "Charan", ["Python", "ML", "SQL"]),  
    ("E104", "Divya", ["Python", "Java", "Cloud"]),  
    ("E105", "Esha", ["Python", "ML", "Cloud"])  
]
```

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

```

17 print('\nPython + ML:')
18 for emp in employees:
19     if "Python" in emp[2] and "ML" in emp[2]:
20         print(emp[1], end=", ")
21 print()
22
23 # 3. SQL but not Java
24 print('\nSQL but not Java:')
25 for emp in employees:
26     if "SQL" in emp[2] and "Java" not in emp[2]:
27         print(emp[1], end=", ")
28 print()
29
30 # 4. Java or Cloud
31 print('\nJava or Cloud:')
32 for emp in employees:
33     if "Java" in emp[2] or "Cloud" in emp[2]:
34         print(emp[1], end=", ")
35 print()
36
37 # 5. Employees with 3 skills
38 print('\nEmployees with 3 skills:')
39 for emp in employees:
40     if len(emp[2]) >= 3:
41         print(emp[1], end=", ")
42 print()
43
44 # 6. Most common skill
45 skill_count = {}
46
47 for emp in employees:
48     for skill in emp[2]:
49         if skill in skill_count:
50             skill_count[skill] += 1
51         else:
52             skill_count[skill] = 1
53
54 most_common = max(skill_count, key=skill_count.get)
55
56 print('\nMost Common Skill:')
57 print(most_common)
58
59 # 7. Employees with at least 3 skills
60 print('\nEmployees with at least 3 skills:')
61 for emp in employees:
62     if len(emp[2]) >= 3:
63         print(emp[1], end=", ")
64 print()
65
66 # 8. Employees without Python
67 print('\nWithout Python:')
68 for emp in employees:
69     if "Python" not in emp[2]:
70         print(emp[1], end=", ")
71 print()
72
73 # 9. Sort employees by number of skills
74 print('\nSorted by Number of Skills:')
75 sorted_employees = sorted(employees, key=lambda x: len(x[2]))
76
77 for emp in sorted_employees:
78     print(emp[1], len(emp[2]))
79
80 # 10. Suitable for Python + ML project
81 print('\nSuitable for Python + ML:')
82 for emp in employees:
83     if "Python" in emp[2] and "ML" in emp[2]:
84         print(emp[1], end=", ")
85 print()

```

Input

stdin input

Output

```

Python:
Arun, Charan, Divya, Esha,

Python + ML:
Arun, Charan, Esha,

```

Run

Save

A bank stores transactions:

```
transactions = [  
    ("T101", "Deposit", 5000),  
    ("T102", "Withdraw", 2000),  
    ("T103", "Deposit", 8000),  
    ("T104", "Withdraw", 1500),  
    ("T105", "Deposit", 3000),  
    ("T106", "Withdraw", 7000)  
]
```

Analyze the transaction history.

Tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than ₹4,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 transactions = [  
2     ('T101', 'Deposit', 5000),  
3     ('T102', 'Withdraw', 2000),  
4     ('T103', 'Deposit', 8000),  
5     ('T104', 'Withdraw', 1500),  
6     ('T105', 'Deposit', 3000),  
7     ('T106', 'Withdraw', 7000)  
8 ]  
9  
10 # Total deposits  
11 total_deposit = 0  
12 total_withdraw = 0  
13  
14 for transaction in transactions:  
15     if transaction[1] == 'Deposit':  
16         total_deposit += transaction[2]  
17     else:  
18         total_withdraw += transaction[2]  
19  
20 print('Total Deposit:', total_deposit)  
21 print('Total Withdrawal:', total_withdraw)  
22  
23 # Net balance  
24 net_balance = total_deposit - total_withdraw  
25 print('Net Balance:', net_balance)  
26  
27 # Largest deposit  
28 deposits = [t[2] for t in transactions if t[1] == 'Deposit']  
29 withdrawals = [t[2] for t in transactions if t[1] == 'Withdraw']  
30  
31 print('\nLargest Deposit:', max(deposits))  
32 print('Largest Withdrawal:', max(withdrawals))  
33  
34 # Transactions greater than 4000  
35 print('\nTransactions > 4000:')  
36 for transaction in transactions:  
37     if transaction[2] > 4000:  
38         print(transaction[0])  
39  
40 # Count deposits and withdrawals  
41 print('\nDeposits:', len(deposits))  
42 print('Withdrawals:', len(withdrawals))  
43  
44 # Average transaction amount  
45 total_amount = sum(t[2] for t in transactions)  
46 average = total_amount / len(transactions)  
47  
48 print('\nAverage Transaction:', format(average, '.2f'))  
49  
50 # Check withdrawals > 50% of deposits  
51 print('\nWithdrawals > 50% of Deposits:')  
52 if total_withdraw > (total_deposit * 0.5):  
53     print('Yes')  
54 else:  
55     print('No')  
56  
57 # Sorted transactions by amount  
58 print('\nSorted Transactions:')  
59 sorted_transactions = sorted(transactions, key=lambda x: x[2])  
60  
61 for transaction in sorted_transactions:  
62     print(transaction[0], transaction[2])
```

Input

Output

```
Total Deposit: 16000  
Total Withdrawal: 10500  
Net Balance: 5500
```

```
Largest Deposit: 8000  
Largest Withdrawal: 7000
```



Run



Save

A system receives:

```
passwords = [  
    "Python@123",  
    "python123",  
    "PYTHON@123",  
    "Py@45",  
    "Python123",  
    "Java#4567",  
    "Admin@2026"  
]
```

A valid password must have:

- At least 8 characters
- Uppercase
- Lowercase
- Digit
- Special character

Analyze:

1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.
5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of case.
8. Sort passwords based on length.
9. Determine the percentage of valid passwords.

Expected Output:

Valid:

```
Python@123  
Java#4567  
Admin@2026
```

Invalid:

```
python123 → No uppercase, No special character  
PYTHON@123 → No lowercase  
Py@45 → Less than 8 characters  
Python123 → No special character
```

Valid Count: 3

Invalid Count: 4

Longest Password:

```
Python@123
```

Python-related passwords:

```
Python@123  
python123  
PYTHON@123  
Python123
```

Valid Percentage: 42.86%

Your Code

```
1 passwords = [  
2     'Python@123',  
3     'python123',  
4     'PYTHON@123',  
5     'Py@45',  
6     'Python123',  
7     'Java#4567',  
8     'Admin@2026'  
9 ]  
10  
11 valid_passwords = []  
12 invalid_passwords = []  
13  
14 # Check all passwords  
15 for password in passwords:  
16     conditions = []  
17  
18     if len(password) < 8:  
19         conditions.append("Less than 8 characters")  
20  
21     if not any(c.isupper() for c in password):  
22         conditions.append("No uppercase")  
23  
24     if not any(c.islower() for c in password):  
25         conditions.append("No lowercase")  
26  
27     if not any(c.isdigit() for c in password):  
28         conditions.append("No digit")  
29  
30     if not any(not c.isalnum() for c in password):  
31         conditions.append("No special character")  
32  
33     if len(conditions) == 0:  
34         valid_passwords.append(password)  
35     else:  
36         invalid_passwords.append((password, conditions))  
37  
38 # Valid passwords  
39 print('Valid:')  
40 for password in valid_passwords:  
41     print(password)  
42  
43 # Invalid passwords  
44 print('\nInvalid:')  
45 for password, conditions in invalid_passwords:  
46     print(password, "->", ", ".join(conditions))  
47  
48 # Counts  
49 print('\nValid Count:', len(valid_passwords))  
50 print('Invalid Count:', len(invalid_passwords))  
51  
52 # Longest password  
53 longest = max(passwords, key=len)  
54 print('\nLongest Password:')  
55 print(longest)  
56  
57 # Passwords containing 'python'  
58 print('\nPython-related passwords:')  
59 for password in passwords:  
60     if 'python' in password.lower():  
61         print(password)  
62  
63 # Sort passwords by length  
64 print('\nPasswords sorted by length:')  
65 for password in sorted(passwords, key=len):  
66     print(password)  
67  
68 # Valid percentage  
69 percentage = (len(valid_passwords) / len(passwords)) * 100  
70 print('\nValid Percentage:', format(percentage, ".2f") + "%")
```

Input

stdin input

Output

```
Valid:  
Python@123  
Java#4567  
Admin@2026
```

Invalid:



Run



Save

Consider the following program:

```
records = [  
    ("Arun", "Python"),  
    ("ARUN", "Python"),  
    ("Bala", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]  
  
names = []  
  
for record in records:  
    name = record[0].strip().lower()  
  
    if name not in names:  
        names.append(name)  
  
print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and their skills.
9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

```
['arun', 'bala', 'charan']
```

Your Code

```
1 records = [  
2     ("Arun", "Python"),  
3     ("ARUN", "Python"),  
4     ("Bala", "Java"),  
5     ("bala", "Java"),  
6     ("Charan", "Python")  
7 ]  
8  
9 names = []  
10  
11 for record in records:  
12     name = record[0].strip().lower()  
13  
14     if name not in names:  
15         names.append(name)  
16  
17 print(names)
```

Input

stdin input

Output

```
['arun', 'bala', 'charan']
```



Run



Save

Consider:

```
text = "data science uses data analysis and data visualization"
```

Analyze the string using Python.

Tasks

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:
(word, frequency)
8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

```
['data', 'science', 'uses', 'analysis', 'and', 'visualization']
```

Frequency:

```
data → 3
```

```
science → 1
```

```
uses → 1
```

```
analysis → 1
```

```
and → 1
```

```
visualization → 1
```

Most Frequent:

```
data
```

Repeated Words:

```
data
```

Longest Word:

```
visualization
```

Your Code

```
1 text = 'data science uses data analysis and data visualization'
2
3 # 1. Convert sentence into list of words
4 words = text.split()
5
6 # 2. Total words
7 print('Total Words:', len(words))
8
9 # 3. Unique words
10 unique_words = []
11 for word in words:
12     if word not in unique_words:
13         unique_words.append(word)
14
15 print('\nUnique Words:')
16 print(unique_words)
17
18 # 4 & 5. Frequency of every word
19 frequency = {}
20
21 for word in words:
22     if word in frequency:
23         frequency[word] += 1
24     else:
25         frequency[word] = 1
26
27 print('\nFrequency:')
28 for word in unique_words:
29     print(word, '->', frequency[word])
30
31 # 6. Most frequently occurring word
32 most_frequent = max(frequency, key=frequency.get)
33 print('\nMost Frequent:')
34 print(most_frequent)
35
36 # 7. Create tuples (word, frequency)
37 word_tuples = []
38
39 for word in unique_words:
40     word_tuples.append((word, frequency[word]))
41
42 # 8. Sort tuples based on frequency
43 sorted_tuples = sorted(word_tuples, key=lambda x: x[1], reverse=True)
44
45 print('\nSorted Tuples:')
46 print(sorted_tuples)
47
48 # 9. Words having frequency greater than 1
49 print('\nWords with Frequency > 1:')
50 for word in unique_words:
51     if frequency[word] > 1:
52         print(word)
53
54 # 10. Longest word
55 longest_word = max(unique_words, key=len)
56 print('\nLongest Word:')
57 print(longest_word)
58
59 # 11. Words beginning with 'd'
60 print('\nWords Beginning with 'd':')
61 for word in unique_words:
62     if word.startswith('d'):
63         print(word)
64
65 # 12. Percentage of repeated words
66 repeated_occurrences = sum(frequency[word] - 1 for word in unique_words)
67 percentage = (repeated_occurrences / len(words)) * 100
68
69 print('\nPercentage of Repeated Words:', format(percentage, '.2f') + '%')
```

Input

stdin input

Output

```
Total Words: 8

Unique Words:
['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:
```



Run



Save

A college maintains:

```
students = [  
    ("A001 ", "ARUN KUMAR", [78, 85, 92]),  
    ("A002", "Bala", [45, 67, 52]),  
    ("a003", "arun kumar", [88, 91, 95]),  
    ("A004", "CHARAN", [72, 76, 81]),  
    ("A005", "Divya ", [55, 48, 62])  
]
```

Each record contains:

(Student ID, Student Name, Marks)

Analyze the complete dataset:

1. Normalize all student IDs using strip() and upper().
2. Normalize all student names using strip() and lower().
3. Calculate total and average marks.
4. Identify students who passed all subjects.
5. Identify students who failed at least one subject.
6. Find the highest total.
7. Find the lowest total.
8. Identify students with average above 75.
9. Detect duplicate student names after normalization.
10. Sort students based on average marks.
11. Find the top two students.
12. Find the subject-wise highest marks.
13. Find the subject-wise average marks.
14. Count the number of students who scored above 80 in at least two subjects.
15. Identify the student who has the highest mark in Subject 3.
16. Explain which data structure is appropriate for student records, marks, and individual student identity, and justify your answer.

Expected Output:

Normalized Records:

```
A001 arun kumar [78, 85, 92]  
A002 bala [45, 67, 52]  
A003 arun kumar [88, 91, 95]  
A004 charan [72, 76, 81]  
A005 divya [55, 48, 62]
```

Totals:

```
arun kumar → 255  
bala → 164  
arun kumar → 274  
charan → 229  
divya → 165
```

Averages:

```
arun kumar → 85.00  
bala → 54.67  
arun kumar → 91.33  
charan → 76.33  
divya → 55.00
```

Passed All Subjects:

```
arun kumar  
arun kumar  
charan
```

Failed At Least One:

```
bala  
divya
```

Highest Total:

```
arun kumar → 274
```

Lowest Total:

```
bala → 164
```

Average > 75:

```
arun kumar  
arun kumar  
charan
```

Duplicate Name:

```
arun kumar
```

Top 2:

```
arun kumar → 91.33  
arun kumar → 85.00
```

Subject-wise Highest:

```
Subject 1 → 88
```

```
Subject 2 → 91
```

arun kumar → 91.33
arun kumar → 85.00

Subject-wise Highest:

Subject 1 → 88

Subject 2 → 91

Subject 3 → 95

Subject-wise Average:

Subject 1 → 67.60

Subject 2 → 73.40

Subject 3 → 78.40

Your Code

```
1 students = [  
2     ('A001', 'ARUN KUMAR', [78, 85, 92]),  
3     ('A002', 'Bala', [45, 67, 52]),  
4     ('A003', 'arun kumar', [88, 91, 95]),  
5     ('A004', 'CHARAN', [72, 76, 81]),  
6     ('A005', 'Divya', [55, 48, 62])  
7 ]  
8  
9 # Normalize ID and name  
10 records = []  
11  
12 for sid, name, marks in students:  
13     records.append((sid.strip().upper(), name.strip().lower(), marks))  
14  
15 print("Normalized Records:")  
16 for sid, name, marks in records:  
17     print(sid, name, marks)  
18  
19 # Totals and averages  
20 print("\nTotals:")  
21 totals = []  
22  
23 for sid, name, marks in records:  
24     total = sum(marks)  
25     totals.append((name, total))  
26     print(name, "->", total)  
27  
28 print("\nAverages:")  
29 averages = []  
30  
31 for sid, name, marks in records:  
32     average = sum(marks) / len(marks)  
33     averages.append((name, average))  
34     print(name, "->", format(average, ".2f"))  
35  
36 # Passed all subjects  
37 print("\nPassed All Subjects:")  
38 for sid, name, marks in records:  
39     if all(mark >= 50 for mark in marks):  
40         print(name)  
41  
42 # Failed at least one subject  
43 print("\nFailed At Least One:")  
44 for sid, name, marks in records:  
45     if any(mark < 50 for mark in marks):  
46         print(name)  
47  
48 # Highest total  
49 highest = max(totals, key=lambda x: x[1])  
50 print("\nHighest Total:")  
51 print(highest[0], "->", highest[1])  
52  
53 # Lowest total  
54 lowest = min(totals, key=lambda x: x[1])  
55 print("\nLowest Total:")  
56 print(lowest[0], "->", lowest[1])  
57  
58 # Average above 75  
59 print("\nAverage > 75:")  
60 for name, average in averages:  
61     if average > 75:  
62         print(name)  
63  
64 # Duplicate names  
65 print("\nDuplicate Name:")  
66 names = [name for sid, name, marks in records]  
67  
68 for name in set(names):  
69     if names.count(name) > 1:  
70         print(name)  
71  
72 # Sort by average  
73 sorted_students = sorted(averages, key=lambda x: x[1], reverse=True)  
74  
75 print("\nSorted by Average:")  
76 for name, average in sorted_students:
```

```

49 highest = max(totals, key=lambda x: x[1])
50 print("\nHighest Total:")
51 print(highest[0], "->", highest[1])
52
53 # Lowest total
54 lowest = min(totals, key=lambda x: x[1])
55 print("\nLowest Total:")
56 print(lowest[0], "->", lowest[1])
57
58 # Average above 75
59 print("\nAverage > 75:")
60 for name, average in averages:
61     if average > 75:
62         print(name)
63
64 # Duplicate names
65 print("\nDuplicate Name:")
66 names = [name for sid, name, marks in records]
67
68 for name in set(names):
69     if names.count(name) > 1:
70         print(name)
71
72 # Sort by average
73 sorted_students = sorted(averages, key=lambda x: x[1], reverse=True)
74
75 print("\nSorted by Average:")
76 for name, average in sorted_students:
77     print(name, "->", format(average, ".2f"))
78
79 # Top 2
80 print("\nTop 2:")
81 for name, average in sorted_students[:2]:
82     print(name, "->", format(average, ".2f"))
83
84 # Subject-wise highest
85 print("\nSubject-wise Highest:")
86 for i in range(3):
87     highest_mark = max(marks[i] for sid, name, marks in records)
88     print("Subject", i + 1, "->", highest_mark)
89
90 # Subject-wise average
91 print("\nSubject-wise Average:")
92 for i in range(3):
93     total = sum(marks[i] for sid, name, marks in records)
94     average = total / len(records)
95     print("Subject", i + 1, "->", format(average, ".2f"))
96
97 # Students scoring above 80 in at least 2 subjects
98 count = 0
99
100 for sid, name, marks in records:
101     if sum(mark > 80 for mark in marks) >= 2:
102         count += 1
103
104 print("\nStudents scoring above 80 in at least two subjects:", count)
105
106 # Highest mark in Subject 3
107 student = max(records, key=lambda x: x[2][2])
108
109 print("\nHighest Mark in Subject 3:")
110 print(student[1], "->", student[2][2])
111
112 # Data structure
113 print("\nData Structure:")
114 print("Student records: List of Tuples")
115 print("Marks: List")
116 print("Student ID and Name: Strings")
117 print("Tuple is immutable, while the outer list is mutable.")

```

Input

stdin input

Output

```

Normalized Records:
A001 arun kumar [78, 85, 92]
A002 bala [45, 67, 52]
A003 arun kumar [88, 91, 95]
A004 charan [72, 76, 81]
A005 divya [56, 48, 62]

```



Run



Save